

Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Fundamentos de Sistemas Computacionales

RADAR 2D

Documento de Atributos de Acreditación — Proyecto 2

Estudiantes

Esteban Alejandro Sánchez Ledezma

Dominick Robles

Profesor: Leonardo Andrés Araya Martínez

I Semestre 2026

Índice

Índice	2
Introducción — Atributo CI	3
Problema 1 — Lecturas erróneas del sensor HC-SR04	4
Identificación del problema	4
Planteamiento de soluciones	4
Justificación técnica de la solución seleccionada	4
Problema 2 — Bloqueo de la GUI por lectura serial bloqueante	5
Identificación del problema	5
Planteamiento de soluciones	5
Justificación técnica de la solución seleccionada	5
Problema 3 — Asociación de lecturas a objetos (tracking multi-objeto)	6
Identificación del problema	6
Planteamiento de soluciones	6
Justificación técnica de la solución seleccionada	6

Introducción — Atributo CI

El presente apartado documenta la aplicación del atributo de acreditación Conocimiento de Ingeniería (CI), definido como la capacidad para aplicar conocimientos a nivel universitario de matemáticas, ciencias naturales, fundamentos de la ingeniería y conocimientos especializados para la solución de problemas complejos de ingeniería.

Durante el desarrollo del Radar 2D se identificaron tres problemas técnicos complejos que requirieron aplicar principios de ingeniería para su resolución. Para cada problema se analizaron al menos tres posibles soluciones y se seleccionó la más adecuada con base en argumentos puramente técnicos.

Problema 1 — Lecturas erróneas del sensor HC-SR04

Identificación del problema

El sensor ultrasónico HC-SR04 utiliza la función `pulseIn()` del Arduino para medir el tiempo de vuelo del eco ultrasónico y calcular la distancia mediante la fórmula:

$$\text{distancia (cm)} = \text{duración}_{\mu\text{s}} \times 0.034 / 2$$

Durante las pruebas se observaron tres fuentes principales de error: (1) ausencia de eco cuando no hay objeto dentro del rango — `pulseIn()` retorna 0 al vencer el timeout —, (2) lecturas de distancia mayores al rango operativo del proyecto (> 100 cm) causadas por reflexiones en paredes lejanas, y (3) falsos positivos generados por ecos secundarios en superficies irregulares o blandas. Sin manejo adecuado de estos casos, el tracker de objetos recibe coordenadas inválidas que degradan la visualización y el cálculo de velocidad.

Planteamiento de soluciones

Se evaluaron tres estrategias para filtrar las lecturas inválidas:

Opción	Descripción	Evaluación
Opción A	Sin filtro: usar todas las lecturas tal como llegan, incluyendo 0 y valores extremos. La GUI descarta el punto si cae fuera del canvas.	✗ Rechazada — El tracker recibe puntos inválidos y crea objetos fantasma.
Opción B	Promedio móvil: acumular N lecturas por ángulo y reportar la media. Reduce el ruido a costa de latencia y complejidad en el firmware.	⚠ Viable pero descartada — Aumenta la latencia del barrido y complica el sketch.
Opción C ✓	Timeout + rango máximo: configurar <code>TIMEOUT_PULSEIN_US=6000</code> μs para limitar <code>pulseIn()</code> ; si retorna 0 o la distancia supera <code>DISTANCIA_MAXIMA_CM=100</code> , reportar -1 y que el software lo descarte.	✓ Seleccionada — Mínima latencia, lógica simple, efectiva para el rango del proyecto.

Justificación técnica de la solución seleccionada

La Opción C fue seleccionada por tres razones técnicas. Primero, el timeout de $6000 \mu\text{s}$ es suficiente para cubrir el rango de 100 cm (el tiempo de vuelo a esa distancia es aproximadamente $2 \cdot 100 / 0.034 \approx 5882 \mu\text{s}$), con un margen de seguridad de $\sim 120 \mu\text{s}$, sin desperdiciar tiempo en esperas innecesarias para objetos fuera del área de interés. Segundo, retornar -1 como centinela permite que el código Python descarte el punto en una sola validación (`if distancia < 0`) antes de entregarlo al tracker, evitando la propagación del error. Tercero, mantener toda la lógica de filtrado en el Arduino reduce el tráfico serial, ya que los valores -1 igualmente se envían pero se descartan en Python inmediatamente, sin afectar la asociación de objetos en `TrackerObjetos`.

Problema 2 — Bloqueo de la GUI por lectura serial bloqueante

Identificación del problema

La función `pyserial.readline()` espera de forma bloqueante hasta recibir una línea completa del Arduino o hasta que venza el `timeout` configurado. Tkinter ejecuta toda su lógica de renderizado, eventos y animaciones en un único hilo principal (`mainloop`). Si `readline()` se ejecuta directamente dentro del `mainloop`, la interfaz se congela durante toda la espera, haciendo que la línea de barrido del radar se detenga y los eventos de usuario (botones, cierre de ventana) queden sin respuesta. Con el Arduino enviando datos a 9600 baudios y un frame de 9 caracteres ("`ángulo,distancia\n`"), la espera máxima es de aproximadamente 10 ms por lectura, pero con el `delay` del servo (40 ms) la GUI se congela hasta 50 ms por ciclo, produciendo un framerate perceptiblemente bajo.

Planteamiento de soluciones

Se evaluaron tres arquitecturas para desacoplar la lectura serial del loop de la GUI:

Opción	Descripción	Evaluación
Opción A	Polling con <code>after()</code> : llamar a <code>readline()</code> directamente desde <code>after()</code> cada N ms con <code>timeout=0</code> en <code>pyserial</code> . Si no hay dato, la función retorna inmediatamente.	⚠ Viable parcialmente — Evita bloqueo completo, pero genera <code>busy-waiting</code> y puede perder datos si N es muy grande.
Opción B	<code>asyncio</code> + <code>tkinter</code> : usar <code>asyncio.get_event_loop()</code> integrado con el <code>mainloop</code> de Tkinter mediante <code>loop.run_until_complete()</code> o <code>asyncio.run()</code> .	✗ Rechazada — La integración <code>asyncio</code> -Tkinter es compleja y no está soportada nativamente en Python 3.9-3.11 sin wrappers adicionales.
Opción C ✓	Hilo daemon + queue. Queue: un Thread separado ejecuta <code>readline()</code> de forma bloqueante y deposita cada lectura en una Queue thread-safe. El <code>mainloop</code> consume la cola periódicamente con <code>after(30, ...)</code> .	✓ Seleccionada — Desacoplamiento total entre I/O serial y renderizado. Cero <code>busy-waiting</code> .

Justificación técnica de la solución seleccionada

La Opción C fue seleccionada por ser la única que cumple simultáneamente los tres requisitos críticos del sistema: (1) no bloquear el `mainloop` de Tkinter bajo ninguna circunstancia, (2) no perder datos del Arduino cuando la GUI está ocupada renderizando, y (3) ser segura para accesos concurrentes. `queue.Queue` está implementada con locks internos en CPython, garantizando que el productor (hilo serial) y el consumidor (hilo GUI) nunca corrompan el estado compartido. Adicionalmente, el hilo se declara como `daemon=True`, lo que asegura que se cierra automáticamente al destruirse la ventana principal, sin necesidad de lógica explícita de limpieza en todos los paths de cierre. El costo en memoria es mínimo: una Queue con capacidad ilimitada que en la práctica nunca supera las 5-10 lecturas en cola.

Problema 3 — Asociación de lecturas a objetos (tracking multi-objeto)

Identificación del problema

El radar entrega una lectura por posición angular: un par (ángulo, distancia) que se convierte en coordenadas cartesianas (x , y). Sin embargo, el requerimiento 02 del proyecto exige rastrear varios objetos simultáneamente y calcular la velocidad individual de cada uno. El problema central es la asociación: dada una nueva lectura (x_2 , y_2), ¿pertenece al mismo objeto que la lectura anterior (x_1 , y_1) o es un objeto diferente? La dificultad aumenta porque el sensor solo detecta el objeto más cercano al eje del haz en cada ángulo, y entre dos barridos consecutivos (~3.6 s) un objeto puede haberse movido varios centímetros.

Planteamiento de soluciones

Se evaluaron tres estrategias de asociación:

Opción	Descripción	Evaluación
Opción A	Asociar por ángulo: si la nueva lectura llega al mismo ángulo que una lectura anterior ($\pm 2^\circ$), se considera el mismo objeto.	 Rechazada — Falla si el objeto se mueve lateralmente entre barridos y aparece en un ángulo diferente.
Opción B	Asociar por distancia radial: si la nueva distancia r coincide con la anterior ($\pm N$ cm), se considera el mismo objeto.	 Rechazada — Imposible con múltiples objetos a distancias similares; no distingue objetos en el mismo rango pero diferente ángulo.
Opción C ✓	Distancia euclidiana en cartesiano: calcular $\sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$ entre la nueva lectura y cada objeto activo. Asociar al más cercano si está dentro de UMBRAL_ASOCIACION_CM=15 cm.	 Seleccionada — Captura el desplazamiento real en el plano 2D, funciona con múltiples objetos simultáneos.

Justificación técnica de la solución seleccionada

La Opción C fue seleccionada porque trabaja directamente en el espacio cartesiano donde se conoce la posición completa del objeto (x , y), en lugar de operar sobre coordenadas parciales (solo ángulo o solo distancia). Al calcular la distancia euclidiana $\sqrt{(\Delta x)^2 + (\Delta y)^2}$ se captura el desplazamiento real del objeto en el plano bidimensional, independientemente de si ese movimiento fue radial, tangencial o una combinación de ambos. El umbral UMBRAL_ASOCIACION_CM = 15 cm fue calibrado experimentalmente: suficientemente grande para tolerar el desplazamiento de un objeto lento entre dos barridos de 3.6 s (~4 cm/s máx), pero suficientemente pequeño para no confundir dos objetos separados en el mismo barrido. Adicionalmente, el timeout TIMEOUT_SEGUNDOS = 3 s permite que objetos que salen del campo del sensor sean eliminados automáticamente, manteniendo limpio el estado del tracker sin intervención del usuario.